Python Cheat Sheet

Basics & Essential Syntax

1. Variables

```
x = 10
name = "Aman"
a, b = 1, 2
```

2. Data Types

```
int       → 10
float     → 3.14
complex   → 1 + 2j
bool      → True, False
str       → "hello"
list      → [1, 2, 3]
tuple     → (1, 2)
dict      → {"a": 1, "b": 2}
set       → {1, 2, 3}
NoneType  → None
```

3. Mutable vs Immutable

```
Mutable   → list, dict set
Immutable → numbe s, bool, str, tuple
            (items may be mutable)
```

4. Type Conversion

```
int("12")   → 12
float("2.5") → 2.5
str(42)     → "42"
bool(0)     → False
```

5. Input and Output

```
input() returns a string.
age = int(input("Age: "))
print("Hi", name, sep=" ", end="\n")
```

6. Arithmetic Operators

```
+  -  *  /  //  %  **
5 + 2  → 7
5 / 2  → 2.5  (true division)
5 // 2 → 2    (floor division)
5 % 2  → 1    (modulus)
2 ** 3 → 8    (power)
```

7. Comparison Operators

```
==   !=   <   <=   >   >=
3 == 3  → True
[illegible] > 2  → True
```

8. Logical Operators

```
and   or   not
True and False → False
not True       → False
```

9. Membership Operators

```
in   not in
"a" in "cat"   → True
3 not in [1, 2] → True
```

10. Identity Operators

```
is   is not
x is None  → True
[] is []   → False
```

11. Equality vs Identity

```
== compares values.
is  compares object identity.
Use x is None to check for None.
```

12. Falsy Values

```
None, False, 0, 0.0, 0j, '', [], (), {}, set()
(empty containers)
```

13. Type Checking

```
type(x) → <class 'int'>
isinstance(x, int) → True
```

14. Comments and Indentation

```
# this is a comment
Indentation defines code blocks (use 4 spaces).
Example:
    if x > 0:
        print("Positive")
```

Python Cheat Sheet

Strings & Formatting

1. Creating & Indexing Strings

```
s = "Python"
s[0] → 'P'
s[-1] → 'n'
```

2. String Slicing

```
s[1:4] → 'yth'
s[::2] → 'Pto'
s[::-1] → 'nohtyP'
End index is excluded.
```

3. Changing Case

```
s.upper() → 'PYTHON'
s.lower() → 'python'
s.title() → 'Python'
s.capitalize() → 'Python'
s.swapcase() → 'pYTHON'
s.casefold() → 'python'
```

4. Trimming Whitespace

```
s = "  Python  "
s.strip() → 'Python'
s.lstrip() → 'Python  '
s.rstrip() → '  Python'
Remove whitespace by default.
```

5. Searching & Counting

```
s = "Python"
s.find("th") → 2
s.index("th") → 2
s.find("z") → -1  (not found)
s.index("z") → ValueError
s.count("o") → 1
```

6. Replacing Substrings

```
s = "Python"
s.replace("Py", "Jy") → "Jython"
```

7. Splitting & Joining

```
s = "a,b,c"
s.split(",") → ['a', 'b', 'c']
text = "line1\nline2\nline3"
text.splitlines() → ['line1', 'line2', 'line3']
"-".join(["a", "b"]) → "a-b"
```

8. Checking Start/End

```
s = "Python"
s.startswith("Py") → True
s.endswith("on") → True
```

9. String Tests

```
s.isalpha()  # only letters
s.isdigit()  # only digits
s.isalnum()  # letters or digits
s.isspace()  # only whitespace
```

10. String Formatting (f-strings)

```
name = "Aman"
f"Hi {name}" → "Hi Aman"
price = 12.5
f"{price::2f}" → "12.50"
```

Escape Sequences

```
"\n" → newline
"\t" → tab
r"\d+" → keeps backslashes (raw string)
```

Strings are Immutable

Methods return new strings.

```
s = s.lower() # reassign to change
```

Python Cheat Sheet

Lists, Tuples & Sequences

1. Lists

```
a = [3, 1, 2]
a[0]   → 3
a[1:]  → [1, 2]
a[::-1] → [2, 1, 3]  # reversed copy
```

2. List Methods

```
a.append(x)      → add one item
a.extend(items)  → add iterable items
a.insert(i, x)   → insert at index i
a.remove(x)      → remove first match
                   (ValueError if absent)
a.pop(i=-1)      → remove and return item
                   (IndexError if invalid)
a.clear()        → remove all items
a.index(x)       → first matching index
                   (ValueError if absent)
a.count(x)       → number of occurrences
a.sort(key=None, reverse=False)
      → sort in place (returns None)
a.reverse()      → reverse in place
a.copy()         → shallow copy

a.sort()    → returns None
sorted(a)   → new sorted list (does not
              modify a)
```

3. Tuples

```
t = (1, 2)
one = (1,)    # single element tuple
t.count(x)    → number of occurrences
t.index(x)    → first matching index
t[0]      → 1
t[0] = 5  → TypeError (no item assignment)
```

4. Unpacking

```
x, y = (1, 2)  → x = 1, y = 2
a, b, c = [1, 2, 3]
first, *rest = [1, 2, 3]
    → first = 1, rest = [2, 3]
x, y = y, x  → swap values
# e.g. x = 1, y = 2  → x = 2, y = 1
```

5. Sequence Tools

```
len(a)    → number of items
x in a    → membership test (True/False)
3 in [1,2,3]  → True
a + b     → concatenation
  [1, 2] + [3, 4] → [1, 2, 3, 4]
a * 2     → repetition
  [1, 2] * 2  → [1, 2, 1, 2]
```

6. Copying Lists

```
b = a     → aliases the same list
  a = [1, 2]
  b = a
  b.append(3)  → a is also [1, 2, 3]
b = a.copy()  → new outer list, but
                nested objects are shared
  a = [[1], [2]]
  b = a.copy()
  b[0].append(99)  → a is also [[1, 99], [2]]
import copy
  b = copy.deepcopy(a)  → recursively copy
                          nested objects

Avoid: rows = [[]] * 3   # shared nested lists
       rows[0].append(1) → all rows change
Use: rows = [[] for _ in range(3)]
          → independent lists
```

Python Cheat Sheet

Dictionaries & Sets

1. Dictionaries (dict)

```
d = {"a": 1, "b": 2}
d["a"]  → 1
d["c"] = 3  → add new key

d.get(k, default)  → safe lookup
  d.get("x", 0)  → 0 (if "x" not found)

d.keys()  → view of keys
d.values()  → view of values
d.items()  → view of (key, value) pairs

d.update(other)  → merge in place
  d.update({"b": 5, "c": 3})

d.setdefault(k, v)  → existing value
                      or insert default
  d.setdefault("x", 0)

d.pop(k, default)  → remove & return
  d.pop("a", None)

d.popitem()  → remove last inserted pair

d.clear()  → remove all items

d.copy()  → shallow copy
  d2 = d.copy()

dict.fromkeys(["x", "y"], 0)  → new dict
      with given keys and value

merged = d | {"z": 4}  → merge (Python 3.9+)
```

Keys must be hashable.
Insertion order is preserved (Python 3.7+).

2. Sets (set)

```
s = {1, 2, 3}
empty_set = set()    # empty set
{} is a dict, not a set

s.add(x)  → add one item
  s.add(4)
s.update(iterable)  → add many items
  s.update([4, 5])
s.remove(x)  → remove item; raises
                KeyError if absent
  s.remove(2)
s.discard(x)  → remove item; no error
                if absent
  s.discard(10)
s.pop()  → remove and return an
           arbitrary item
  x = s.pop()
s.clear()  → remove all items
s.copy()  → shallow copy
  s2 = s.copy()
```

3. Set Operations

```
s | t   → union (all items)
s & t   → intersection (common items)
s - t   → difference (items in s, not in t)
s ^ t   → symmetric difference
s <= t  → subset (all s in t)
s >= t  → superset (all t in s)
s.isdisjoint(t)  → no shared items
```

Named methods:
`s.union(t)`, `s.intersection(t)`,
`s.difference(t)`, `s.symmetric_difference(t)`,
`s.issubset(t)`, `s.issuperset(t)`

4. Other

```
frozenset(iterable)  → immutable set
```

Sets have unique hashable items.
No indexing. No guaranteed iteration order.

Python Cheat Sheet

Control Flow & Processing

1. Conditions

```
if x > 0:
    result = "positive"
elif x == 0:
    result = "zero"
else:
    result = "negative"

# Ternary conditional expression
result = "yes" if ok else "no"
```

2. For Loop

```
for n in range(3):
    print(n)   # 0, 1, 2

range(start, stop, step)
# stop is excluded
range(1, 5, 2)  → 1, 3
```

3. While Loop

```
while condition:
    # repeat while condition is True
    ...
```

4. Loop Control

```
break    → exit the loop
continue → skip to next iteration
pass     → placeholder (does nothing)
```

5. Loop Else

else runs only if no break (including zero iterations).

```
for n in range(3):
    print(n)
else:
    print("completed")  # runs if no break
```

6. Comprehensions

```
List comprehension:
  [x*x for x in range(5) if x % 2 == 0]
   → [0, 4, 16]
Set comprehension:
  {x*x for x in nums}
Dictionary comprehension:
  {x: x*x for x in nums}
```

7. Useful Iteration Tools

```
enumerate(["a", "b"], start=1)
   → (1, "a"), (2, "b")
zip(names, scores)
    -> pairs items, stops at shortest
   list(zip(["a", "b"], [10, 20, 30]))
    → [("a", 10), ("b", 20)]
reversed(seq)
   → reverse iterator
   list(reversed([1, 2, 3])) → [3,2,1]
```

8. Sorting

```
sorted(words, key=len) → sort by length
sorted(nums, reverse=True) → descending
```

9. Common Built-in Functions

```
len(seq)  → length
sum(nums) → sum of numbers
min(nums), max(nums) → minimum, maximum
abs(-3) → 3
round(2.345, 2) → 2.35
any([False, True]) → True
all([True, True]) → True
all([]) = True;  any([]) = False
```

10. Match-Case (Python 3.10+)

Structural pattern matching.

```
match value:
    case 1:
        ...
    case _:
        ...
```

Python Cheat Sheet

Functions, Scope & Generators

1. Defining Functions

```
def add(a: int, b: int = 0) → int:
    return a + b
add(2, 3)   → 5
add(a=2, b=3)   → 5
```

Type hints don't enforce runtime types; they are for readability and tools.

2. *args and **kwargs

```
*args   → collects extra positional arguments as a tuple
**kwargs → collects extra keyword arguments as a dict

def f(*args, **kwargs):
    ...
f(1, 2, 3)  → args = (1, 2, 3)
f(x=1, y=2) → kwargs = {'x': 1, 'y': 2}
f(*values, **options)  → unpacks iterables/dicts
```

3. Lambda & Higher-Order Functions

```
lambda x: x * x  → anonymous function
sorted(pairs, key=lambda p: p[1])
    → sort by second element
map(fn, items)  → returns an iterator
filter(fn, items)  → returns an iterator
list(map(lambda x: x*x, [1, 2, 3]))
   → [1, 4, 9]
```

4. Variable Scope (LEGB)

LEGB rule for name lookup:

- L → Local (inside current function)
- E → Enclosing (in outer function)
- G → Global (in module)
- B → Built-in (Python built-ins)

5. global and nonlocal

```
global x  → update module-level binding
x = 0
def set_x():
    global x
    x = 10
nonlocal x  → update enclosing function binding
def outer():
    x = 0
    def inner():
        nonlocal x
        x = 5
```

6. Mutable Default Arguments

Avoid using mutable defaults.
Use None and create inside function.

```
def collect(x, items=None):
    if items is None:
        items = []
    items.append(x)
    return items
collect(1)  → [1]
collect(2)  → [2]  # new list each call
```

7. Iterators and next()

```
it = iter([10, 20])  → iterator object
next(it)  → 10
next(it)  → 20
next(it, None)  → None  # fallback when exhausted
```

8. Generators

```
def squares(n):
    for i in range(n):
        yield i * i
(x * x for x in range(5))  → generator expression (lazy)
list(squares(4))  → [0, 1, 4, 9]
```

9. Decorators

```
@decorate
def f():
    ...
```
→ same as `f = decorate(f)`

Use functools.wraps in wrapper decorators to preserve metadata (e.g., __name__, __doc__).

Python Cheat Sheet

Classes, Exceptions & Files

1. Defining a Class

```
class User:
    species = "human"  # class attribute
    def __init__(self, name):
        self.name = name  # instance attribute
    def greet(self):
        return f"Hi {self.name}"
    def __repr__(self):
        return f"User({self.name!r})"
u = User("Aman")
u.greet()  → "Hi Aman"
```

2. Inheritance

```
class Admin(User):  # inherits from User
    pass
# call parent initializer
class Admin(User):
    def __init__(self, name):
        super().__init__(name)
```

3. Properties

```
class User:
    def __init__(self, name):
        self._name = name
    @property  # exposes method as attribute
    def name(self):
        return self._name
u = User("Aman")
u.name  → "Aman"
```

4. Exceptions

```
try:
    n = int(text)
except ValueError:
    n = 0
else:
    print("valid")
finally:
    print("done")
raise ValueError("Invalid input")  # raise an error
```

5. Context Manager (Files)

```
with open("notes.txt", "r", encoding="utf-8") as f:
    text = f.read()  # file auto closes
```

6. File Modes

```
"r"  → read (default, error if not found)
"w"  → write (truncate if exists)
"a"  → append (create if not exists)
"b"  → binary mode (e.g. "rb", "wb")
"+"  → read and write (e.g. "r+", "w+")
```

7. Reading & Writing Files

```
f.read()  → read entire file (string)
f.read(n)  → read up to n chars (text mode)
f.readline()  → read next line (string)
for line in f:  → memory-efficient iteration
    for line in f:
        print(line.strip())
f.write(text)  → write a string
f.writelines(lines)  → write multiple strings;
                       no separators added

# example: write to a file
with open("out.txt", "w", encoding="utf-8") as f:
    f.write("Hello\n")
```

Python Cheat Sheet

Modules & Useful Tools

1. Importing Modules

```
import math
import datetime as dt
from pathlib import Path
from collections import Counter, defaultdict, deque
```

2. Script Entry Point

```
def main():
    print("Hello")
if __name__ == "__main__":
    main()
```

3. math Module

```
math.sqrt(16)  → 4.0
math.ceil(2.2) → 3
math.floor(2.8) → 2
```

4. random Module

```
import random
random.randint(1, 6) → 1 to 6 (inclusive)
random.choice(seq)   → random element
random.sample(seq, 2) → 2 elements (no replacement)
# for secure randonmess, use secrets
  import secrets
  secrets.choice(seq) → secure choice
```

5. datetime Module

```
import datetime as dt
now = dt.datetime.now()
now.strftime("%Y-%m-%d") → date formatted
                           as YYYY-MM-DD
```

6. pathlib Module

```
from pathlib import Path
p = Path("a.txt")
p.read_text(encoding="utf-8") → read file
p.write_text("Hi", encoding="utf-8") → write file
```

7. JSON (json Module)

```
import json
json.loads(text)  → JSON to Python
json.dumps(obj)   → Python to JSON
json.dumps(obj, indent=2) → pretty formatting
```

8. Regular Expressions (re Module)

```
import re
re.search(r"\d+", text)  → match or None
re.sub(r"\s+", " ", text) → replace whitespace
```

9. collections Module

```
from collections import Counter, defaultdict, deque
Counter("aba")  → {'a': 2, 'b': 1}
d = defaultdict(list) → default empty lists
d["a"].append(1)
dq = deque()    → double-ended queue
dq.append(1)    → add right
dq.appendleft(0) → add left
dq.pop()        → remove right
dq.popleft()    → remove left
```

10. Useful Built-in Functions

```
help(obj)          → show help for object
dir(obj)           → list attributes
type(obj)          → get type
isinstance(obj, T) → type check
breakpoint()       → start debugger (pdb)
```

11. Terminal Commands (not Python)

```
# create virtual environment
python -m venv .venv
# activate (Linux/macOS)
source .venv/bin/activate
# activate (Windows cmd)
.venv\Scripts\activate.bat
# install package
python -m pip install package
```